```
   acts_2, act_means2, qvals2, obs_small_2, ids_2 = \
   team_step(team2,params[0],acts_2,layers)
        env.set_action(team2,
acts_2.detach().numpy().astype(np.int32))

        done = env.step()
4

        _, _, qnext1, _, ids_1 =
team_step(team1,params[0],acts_1,layers)  5
        _, _, qnext2, _, ids_2 =
team_step(team2,params[0],acts_2,layers)

        env.render()
6

        rewards1 = torch.from_numpy(env.get_reward(team1)).float()
7
        rewards2 = torch.from_numpy(env.get_reward(team2)).float()
```

- ***1*** While the game is not over
- ***2*** Uses the team_step method to collect environment data and choose actions for the agents using the DQN
- ***3*** Instantiates the chosen actions in the environment
- ***4*** Takes a step in the environment, which will generate a new observation and rewards
- ***5*** Reruns team_step to get the Q values for the next state in the environment
- ***6*** Renders the environment for viewing later
- ***7*** Collects the rewards into a tensor for each agent

The `while` loop runs as long as the game is not over; the game ends when all the agents on one team die. Within the `team_step` function, we first get the observation tensor and subset the part we want as we described before, resulting in a 13 × 13 × 2 tensor. We also get `ids_1`, which are the indices for the agents that are still alive on team 1. We also need to get the coordinate positions of each agent on each team. Then we use our `infer_acts` function to choose actions for each agent and instantiate them in the environment, and finally take an environment step, which will generate new observations and rewards. Let’s continue in the while loop.

**Listing 9.22. Adding to the replay (still in `while loop` from listing 9.21)**